

Assignment Operators

The following table lists the assignment operators supported by the C language:

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	$C = A + B$ will assign the value of $A + B$ to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator.	$C <<= 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator.	$C >>= 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator.	$C \&= 2$ is same as $C = C \& 2$
^=	Bitwise exclusive OR and assignment operator.	$C \wedge= 2$ is same as $C = C \wedge 2$
=	Bitwise inclusive OR and assignment operator.	$C = 2$ is same as $C = C 2$

Example

Try the following example to understand all the assignment operators available in C:

```
#include<stdio.h>

main() {
    int a =21;
    int c ;

    c = a;
    printf("Line 1 - = Operator Example, Value of c = %d\n", c );

    c += a;
    printf("Line 2 - += Operator Example, Value of c = %d\n", c );

    c -= a;
    printf("Line 3 - -= Operator Example, Value of c = %d\n", c );

    c *= a;
    printf("Line 4 - *= Operator Example, Value of c = %d\n", c );

    c /= a;
    printf("Line 5 - /= Operator Example, Value of c = %d\n", c );

    c =200;
    c %= a;
    printf("Line 6 - %= Operator Example, Value of c = %d\n", c );

    c <<=2;
    printf("Line 7 - <<= Operator Example, Value of c = %d\n", c );

    c >>=2;
    printf("Line 8 - >>= Operator Example, Value of c = %d\n", c );

    c &=2;
    printf("Line 9 - &= Operator Example, Value of c = %d\n", c );

    c ^=2;
    printf("Line 10 - ^= Operator Example, Value of c = %d\n", c );

    c |=2;
    printf("Line 11 - |= Operator Example, Value of c = %d\n", c );
}
```

When you compile and execute the above program, it produces the following result:

```
Line 1 - = Operator Example, Value of c = 21
```

```

Line 2 - += Operator Example, Value of c = 42
Line 3 - -= Operator Example, Value of c = 21
Line 4 - *= Operator Example, Value of c = 441
Line 5 - /= Operator Example, Value of c = 21
Line 6 - %= Operator Example, Value of c = 11
Line 7 - <=> Operator Example, Value of c = 44
Line 8 - >>= Operator Example, Value of c = 11
Line 9 - &= Operator Example, Value of c = 2
Line 10 - ^= Operator Example, Value of c = 0
Line 11 - |= Operator Example, Value of c = 2

```

Operators Precedence in C

Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has a higher precedence than the addition operator. For example, $x = 7 + 3 * 2$; here, x is assigned 13, not 20 because operator $*$ has a higher precedence than $+$, so it first gets multiplied with $3*2$ and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	<<= >>=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right

Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Example

Understand the following example to understand operator precedence in C:

```
#include<stdio.h>

main() {
    int a =20;
    int b =10;
    int c =15;
    int d =5;
    int e;

    e =(a + b)* c / d; // ( 30 * 15 ) / 5
    printf("Value of (a + b) * c / d is : %d\n", e );

    e =((a + b)* c) / d; // (30 * 15 ) / 5
    printf("Value of ((a + b) * c) / d is : %d\n", e );

    e =(a + b)*(c / d); // (30) * (15/5)
    printf("Value of (a + b) * (c / d) is : %d\n", e );

    e = a +(b * c) / d; // 20 + (150/5)
    printf("Value of a + (b * c) / d is : %d\n", e );

    return 0;
}
```

When you compile and execute the above program, it produces the following result:

```
Value of (a + b) * c / d is : 90
Value of ((a + b) * c) / d is : 90
Value of (a + b) * (c / d) is : 90
Value of a + (b * c) / d is : 50
```

5.7 Decision Making

Decision making structures require that the programmer specifies one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed. If the condition is determined to be true, some specific statements are executed and if false, then other set of statements are executed.

Shown below is the general form of a typical decision-making structure found in most of the programming languages:

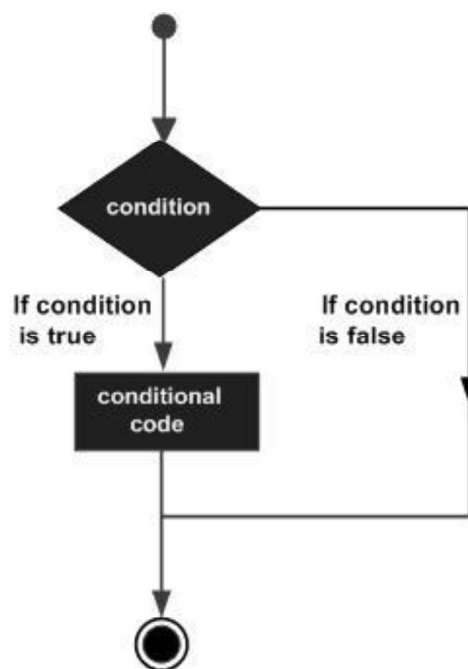


Fig. 5.8 Flowchart of Decision Making in C

C programming language assumes any **non-zero** and **non-null** values as **true**, and if it is either **zero** or **null**, then it is assumed as **false** value. C programming language provides the following types of decision-making statements:

Sr.No.	Statement & Description
1	<u>if statement</u> An if statement consists of a boolean expression followed by one or more statements.
2	<u>if...else statement</u> An if statement can be followed by an optional else statement , which executes when the Boolean expression is false.

3	<u>nested if statements</u> You can use one if or else if statement inside another if or else if statement(s).
4	<u>switch statement</u> A switch statement allows a variable to be tested for equality against a list of values.
5	<u>nested switch statements</u> You can use one switch statement inside another switch statement(s).

If-else Statement

An **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is false.

Syntax

The syntax of an **if...else** statement in C programming language is:

```
if(boolean_expression) {
    /* statement(s) will execute if the boolean expression is true
*/
} else {
    /* statement(s) will execute if the boolean expression is false
*/
}
```

If the Boolean expression evaluates to **true**, then the **if block** will be executed, otherwise, the **else block** will be executed.

C programming language assumes any **non-zero** and **non-null** values as **true**, and if it is either **zero** or **null**, then it is assumed as **false** value.

Example

```
#include<stdio.h>

int main (){

/* local variable definition */
int a =100;

/* check the boolean condition */
if( a <20){
/* if condition is true then print the following */
```

```

        printf("a is less than 20\n");
    }else{
        /* if condition is false then print the following */
        printf("a is not less than 20\n");
    }

    printf("value of a is : %d\n", a);

return 0;
}

```

When the above code is compiled and executed, it produces the following result:

```

a is not less than 20;
value of a is : 100

```

Nested If Statements

It is always legal in C programming to **nest** if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax

The syntax for a **nested if** statement is as follows:

```

if( boolean_expression 1) {

    /* Executes when the boolean expression 1 is true */
    if(boolean_expression 2) {
        /* Executes when the boolean expression 2 is true */
    }
}

```

You can nested **else if...else** in the similar way as you have nested *if* statements.

Example

```

#include<stdio.h>

int main (){

    /* local variable definition */
    int a =100;
    int b =200;

    /* check the boolean condition */
    if( a ==100){

```

```

/* if condition is true then check the following */
if( b ==200){
/* if condition is true then print the following */
    printf("Value of a is 100 and b is 200\n");
}
}

    printf("Exact value of a is : %d\n", a );
    printf("Exact value of b is : %d\n", b );

return0;
}

```

When the above code is compiled and executed, it produces the following result:

```

Value of a is 100 and b is 200
Exact value of a is : 100
Exact value of b is : 200

```

Switch Statements

A **switch** statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each **switch** case.

Syntax

The syntax for a **switch** statement in C programming language is as follows:

```

switch(expression){

case constant-expression :
    statement(s);
break; /* optional */

case constant-expression :
    statement(s);
break; /* optional */

/* you can have any number of case statements */
default: /* Optional */
    statement(s);
}

```

The following rules apply to a **switch** statement:

- The **expression** used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The **constant-expression** for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
- When a **break** statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a **break**. If no **break** appears, the flow of control will *fall through* to subsequent cases until a break is reached.
- A **switch** statement can have an optional **default** case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No **break** is needed in the default case.

Example

```
#include<stdio.h>

int main () {

    /* local variable definition */
    char grade = 'B';

    switch(grade){
    case 'A':
        printf("Excellent!\n");
    break;
    case 'B':
    case 'C':
        printf("Well done\n");
```

```

break;
case 'D':
    printf("You passed\n");
break;
case 'F':
    printf("Better try again\n");
break;
default:
    printf("Invalid grade\n");
}

    printf("Your grade is  %c\n", grade );

return 0;
}

```

When the above code is compiled and executed, it produces the following result:

```

Well done
Your grade is B

```

5.8 Loops in C

Loops are used when a block of code needs to be executed several times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. Programming languages provide various control structures that allow for more complicated execution paths.

C programming language provides the following types of loops to handle looping requirements.

Loop Type & Description
<u>while loop</u> Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
<u>for loop</u> Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
<u>do...while loop</u> It is more like a while statement, except that it tests the condition at the end of the loop body.
<u>nested loops</u> You can use one or more loops inside any other while, for, or do..while loop.

While Loop

A **while** loop in C programming repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a **while** loop in C programming language is:

```
while(condition) {  
    statement(s); }
```

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any expression, and true is any nonzero value. The loop iterates while the condition is true.

When the condition becomes false, the program control passes to the line immediately following the loop.

Example

```
#include<stdio.h>  
  
int main () {  
  
    /* local variable definition */  
    int a =10;  
  
    /* while loop execution */  
    while( a <20){  
        printf("value of a: %d\n", a);  
        a++;  
    }  
  
    return0;  
}
```

When the above code is compiled and executed, it produces the following result:

```
value of a: 10  
value of a: 11  
value of a: 12  
value of a: 13  
value of a: 14  
value of a: 15  
value of a: 16  
value of a: 17  
value of a: 18  
value of a: 19
```

For Loop

A **for** loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

Syntax

The syntax of a **for** loop in C programming language is:

```
for ( init; condition; increment ) {  
    statement(s);  
}
```

Below is the flow of control in a 'for' loop –

- The **init** step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and the flow of control jumps to the next statement just after the 'for' loop.
- After the body of the 'for' loop executes, the flow of control jumps back up to the **increment** statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the 'for' loop terminates.

Example

```
#include<stdio.h>  
  
int main () {  
  
    int a;  
  
    /* for loop execution */  
    for( a =10; a <20; a = a +1){
```

```

        printf("value of a: %d\n", a);
    }
    return 0;
}

```

When the above code is compiled and executed, it produces the following result:

```

value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19

```

Loop Control Statements

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. C supports the following control statements.

Control Statement & Description
<u>break statement</u> Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
<u>continue statement</u> Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
<u>goto statement</u> Transfers control to the labeled statement.

The **break** statement in C programming has the following two usages –

- When a **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- It can be used to terminate a case in the **switch** statement (covered in the next chapter).

If you are using nested loops, the **break** statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax

The syntax for a **break** statement in C is as follows:

```
break;
```

5.9 Functions in C

A function is a group of statements that together perform a task. Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions. You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task.

A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

The C standard library provides numerous built-in functions that your program can call. For example, **strcat()** to concatenate two strings, **memcpy()** to copy one memory location to another location, and many more functions.

A function can also be referred as a method or a sub-routine or a procedure, etc.

Defining a Function

The general form of a function definition in C programming language is as follows:

```
return_type function_name( parameter list ){  
    body of the function  
}
```

The following are the parts of a function –

- **Return Type** – A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the **return_type** is the keyword **void**.
- **Function Name** – This is the actual name of the function. The function name and the parameter list together constitute the function signature.

- **Parameters** – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** – The function body contains a collection of statements that define what the function does.

Example

Given below is the source code for a function called **max()**. This function takes two parameters num1 and num2 and returns the maximum value between the two.

```
/* function returning the max between two numbers */
int max(int num1,int num2){

/* local variable declaration */
int result;

if(num1 > num2)
    result = num1;
else
    result = num2;

return result;
}
```

Function Declarations

A function **declaration** tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately. A function declaration has the following parts:

return_type function_name(parameter list);

For the above defined function max(), the function declaration is as follows –

int max(int num1, int num2);

Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration:

int max(int, int);

Function declaration is required when you define a function in one source file and you call that function in another file. In such case, you should declare the function at the top of the file calling the function.

Calling a Function

To use a function, you will have to call that function to perform the defined task. When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

To call a function, you simply need to pass the required parameters along with the function name, and if the function returns a value, then you can store the returned value. For example:

```
#include<stdio.h>

/* function declaration */
int max(int num1,int num2);

int main (){

/* local variable definition */
int a =100;
int b =200;
int ret;

/* calling a function to get max value */
ret = max(a, b);

printf("Max value is : %d\n", ret );

return0;
}

/* function returning the max between two numbers */
int max(int num1,int num2){

/* local variable declaration */
int result;

if(num1 > num2)
    result = num1;
```



```
else
    result = num2;
return result;
}
```

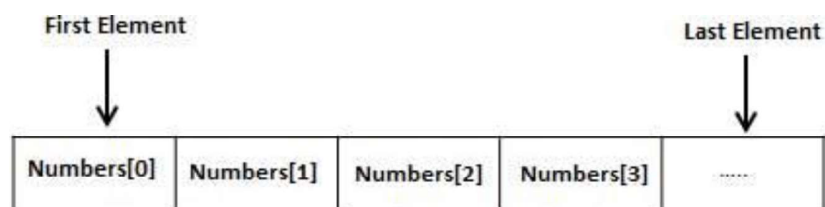
We have kept max() along with main() and compiled the source code. While running the final executable, it would produce the following result:

Max value is : 200

5.10 Arrays in C

Arrays are a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type. Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



Declaring Arrays

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows:

```
type arrayName [ arraySize ];
```

This is called a *single-dimensional* array. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type. For example, to declare a 10-element array called **balance** of type double, use this statement:

```
double balance[10];
```

Here *balance* is a variable array which is sufficient to hold up to 10 double numbers.

Initializing Arrays

You can initialize an array in C either one by one or using a single statement as follows:

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [].

If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write:

```
double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

You will create exactly the same array as you did in the previous example. Following is an example to assign a single element of the array:

```
balance[4] = 50.0;
```

The above statement assigns the 5th element in the array with a value of 50.0. All arrays have 0 as the index of their first element which is also called the base index and the last index of an array will be total size of the array minus 1. Shown below is the pictorial representation of the array we discussed above:

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

Accessing Array Elements

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example:

```
double salary = balance[9];
```

The above statement will take the 10th element from the array and assign the value to salary variable. The following example Shows how to use all the three above mentioned concepts viz. declaration, assignment, and accessing arrays:

```
#include<stdio.h>

int main () {

int n[10]; /* n is an array of 10 integers */
int i,j;
```

```

/* initialize elements of array n to 0 */
for( i =0; i <10; i++){
    n[ i ]= i +100; /* set element at location i to i + 100 */
}

/* output each array element's value */
for(j =0; j <10; j++){
    printf("Element[%d] = %d\n", j, n[j]);
}

return0;
}

```

When the above code is compiled and executed, it produces the following result:

```

Element[0] = 100
Element[1] = 101
Element[2] = 102
Element[3] = 103
Element[4] = 104
Element[5] = 105
Element[6] = 106
Element[7] = 107
Element[8] = 108
Element[9] = 109

```

5.11 Strings in C

Strings are actually one-dimensional array of characters terminated by a **null** character '\0'. Thus, a null-terminated string contains the characters that comprise the string followed by a **null**.

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

If you follow the rule of array initialization then you can write the above statement as follows:

```
char greeting[] = "Hello";
```

There is no need to place the *null* character at the end of a string constant. The C compiler automatically places the '\0' at the end of the string when it initializes the array. Let us try to print the above mentioned string:

```
#include<stdio.h>

int main () {

char greeting[6]={'H','e','l','l','o','\0'};
    printf("Greeting message: %s\n", greeting );
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

Greeting message: Hello

Teacher Notes:

- Inbuilt string handling functions can also be taught i.e., strcpy and strtok etc.

Exercise

Select the most appropriate option.

1. Which of the following characters are used to comment in C.
a. \b. /* */ c. % d. Both a and b
2. Which one of the following is a format specifier of *char*.
a. %db.&l c. %c d. Both a and b
3. Which of the following functions is used to take inputs from users.
a. sprintf() b. printf() c. scanf() d. None of these
4. Which one of the following is a format specifier of *string*.
a. %s b. &l c. %c d. Both a and b
5. Which one of the following is a format specifier of *int*.
a. %d b. &l c. %c d. Both a and b

Answer the following question.

1. Write a program to take input of name, rollno and marks obtained by a student in 5 subjects. Each subject have 100 full marks. Display the name, roll no with percentage score secured.
2. Write a program to add two numbers (5&7) and display its sum.
3. Write a program to input marks of 5 subjects (Physics, Chemistry, Math, English & Biology) for a student. Display the rank of each subjects and also the result of total marks and percentage obtained with his/her rank in the class. The rank is categorized as fail (marks < 40%), pass & third division (marks between 40 to

55%), second (marks between 55 to 65%), first (marks between 65 to 80%), Distinction (marks between 80 to 95%), extra ordinary (marks above 95 to 100%).

4. Write a program to input two integer numbers and display the sum of even numbers between these two input numbers.
5. Write a program to add, subtract, multiply and divide two integers using user defined type function with return type.
6. Write a program to find separately the sum of the positive and negative integer elements of an array of size 10. Define a function called sortarray(int[]), pass the array to this function and display the array elements into ascending order.
7. Write a program to read a sentence and count the number of characters & words in that sentence.